

CS 354 Fall 2025

Lab 6: Multiple Producers and Multiple Consumers Interacting Across A Bounded Buffer [20 pts]

Due: 11/1/2025 (Sat), 11:59 PM

Objectives

Understand how semaphores can coordinate multiple producer and consumer processes sharing a bounded (circular) buffer, and how to use semaphores to control concurrent access safely and efficiently.

What You Will Learn

You will implement a bounded buffer system that allows one to create a bounded buffer, allow processes to use it, deallocate the buffer, and then use the same data structure to create a new buffer, and so on.

Readings

Chapter 6, 7 from the XINU textbook.

Source code

Start from a fresh copy of Xinu `xinu-fall2025.tar.gz` by running the command:

```
tar zxvf /homes/cs354/xinu-fall2025.tar.gz
```

The Problem To Be Solved

1. Listen to the TAs discuss how a serially-reusable resource operates, and how concurrent access to a bounded buffer works with multiple producer and consumer processes. TAs will also explain some of the requirements for your assignment.
2. Declare a global struct, `bb`, that your bounded buffer functions will use. The `bb` struct must contain a state variable that says whether a buffer is currently active, a pointer to the circular buffer itself, and any other data items your bounded buffer functions need (e.g., a variable that tells the next slot in the buffer for a consumer to use). Have initial values set so the buffer is inactive.

Note: the TAs will discuss a few options and details that will help you decide how to define the `bb` struct.

3. Write five functions that form a bounded buffer mechanism as follows:

`int32 bb_create(int32 bufsize)` in `system/bb_create.c`

Allocates the actual buffer with `bufsize` entries, marks the state of the buffer active, and initializes other variables in the bounded buffer struct, `bb`. Returns `YSERR` if the buffer is already active or any other error occurs.

`int32 bb_in(byte b)` in `system/bb_in.c`

Blocks until space is available in the buffer, and then inserts one byte of data, `b`. Returns `YSERR` if any errors occur (e.g., the buffer is inactive) or `OK` if successful.

`int32 bb_end()` in `system/bb_end.c`

Marks the end of the data being sent. Returns `OK` or `YSERR`. The call to `bb_end` may block if the buffer is full; the details of how to mark the end of data is up to you. However, once `bb_end` has been called, all further attempts to insert data will return `YSERR`.

`int32 bb_out()` in `system/bb_out.c`

Blocks until the buffer contains at least one byte of data or the end of data has been set by `bb_end()`. Returns `EOF` if the end of data has been reached.

Otherwise, `bb_out` extracts and returns the next byte of data from the bounded buffer. The call returns `YSERR` if any errors occur (e.g., the buffer is inactive).

`int32 bb_delete()` in `system/bb_delete.c`

Delete the current bounded buffer: free the actual buffer that was allocated by `bb_create`, allow any waiting processes to execute, and free other resources that were allocated. Essentially, return the `bb` struct to the initial state of inactive, ready to be reused. Release any processes blocked by `bb_in()`, `bb_out()`, or `bb_end` (if your `bb_end` blocks processes). Arrange for calls to bounded buffer functions that are terminated by `bb_delete` to return `YSERR`.

4. Test your bounded buffer

Single producer and single consumer

Test all possible 8-bit data values

Single-producer, multiple consumers

Multiple-producers, multiple consumers

Submission Instructions

General instructions:

When implementing code in the labs, please maintain separate versions/copies of code so that mistakes such as unintentional overwriting or deletion of code is prevented. This is in addition to the efficiency that such organization provides. You may use any number of version control systems such as GIT and RCS. Please make sure that your code is protected from public access. For example, when using GIT, use git that manages code locally instead of its on-line counterpart github. If you prefer not to use version control tools, you may just use manual copy to keep track of different versions required for development and testing. More vigilance and discipline may be required when doing so.

You must submit a directory named `lab6` that contains your entire Xinu directory:

```
lab6
```

```
|--- xinu-fall2025
```

Electronic turn-in instructions:

- 1) Please make sure to **follow the file and function naming convention as mentioned** in the lab handout.
- 2) Make sure there are **no dependencies** to your main.c file as we will replace this during grading.
- 3) Go to the `xinu-fall2025/compile` directory and run: `make clean`.
- 4) Go to the directory where `lab6` (lab directory containing `xinu-fall2025/`) is a subdirectory.
 - a) For example, if `/homes/alice/cs354/lab6/xinu-fall2025` is your directory structure, go to `/homes/alice/cs354`.
- 5) Type the following command

```
turnin -c cs354 -p lab6 lab6
```

You can check/list the submitted files using

```
turnin -c cs354 -p lab6 -v
```

Please make sure to disable all debugging output before submitting your code.